

Data Science for Innovation

W10: Unstructured Data—Natural Language Processing Naïve Bayes

Presented by

Ali Anaissi

Today: Unstructured data

Objective

Learn machine learning tools in Python for text categorization and forecasting.

Lecture

- Text pre-processing
- Naïve Bayes
- Text-driven forecasting

Readings

- Data Science from Scratch, Ch. 13
- Doing Data Science, Ch. 4

Exercises

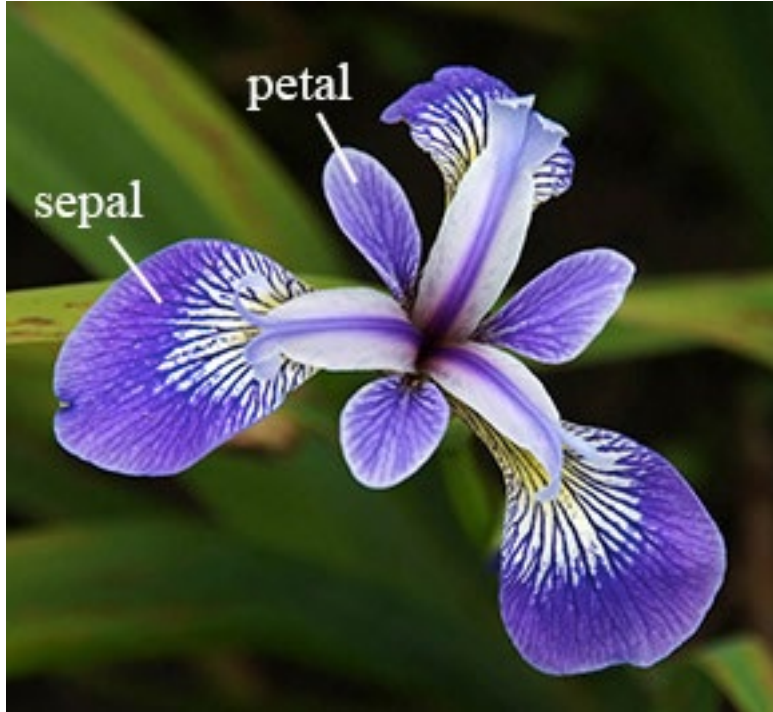
- Spam detection
- Predicting box office returns
- Information extraction

Unstructured data refers to information that does not have a pre-defined data model.

Unstructured data is typically text-heavy, but may contain dates, numbers and facts as well.

This results in ambiguities that make it more difficult to understand than data in structured databases.

Structured data



- Fielded data
- Stored in databases
- E.g.:
 - Sensor data
 - Financial data
 - Click streams
 - Measurements

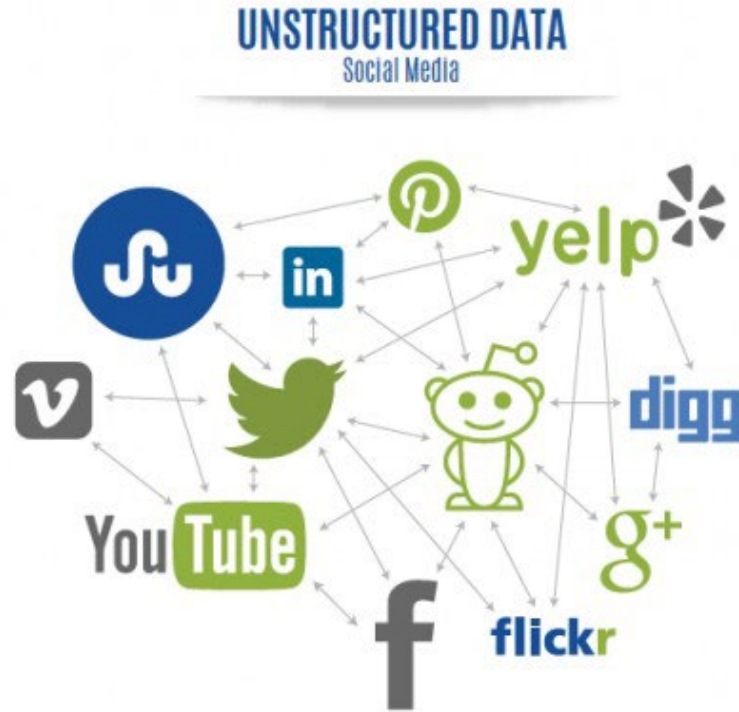
Unstructured data

“In the history of cinematic mustaches, few have been as disgusting as that of Rye Gerhardt (Kieran Culkin), the youngest scion of North Dakota’s reigning crime family and the stray spark that sets off the powder-keg second season of Fargo.”

- 80-90% of all potentially usable business information
- E.g.:
 - Images
 - Video
 - Email
 - Social media

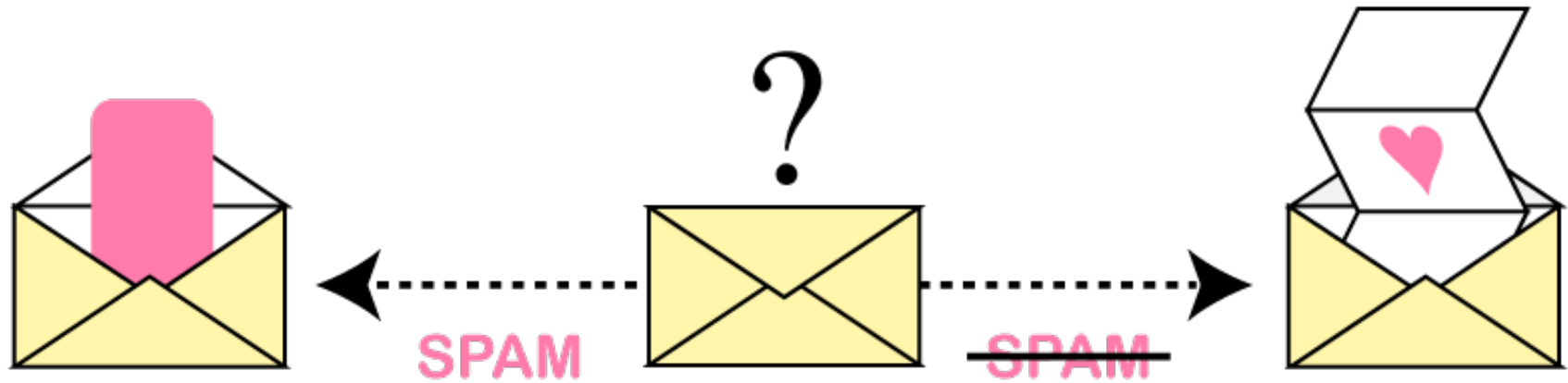
(From a Slate review of Fargo Season 2)

Social media data



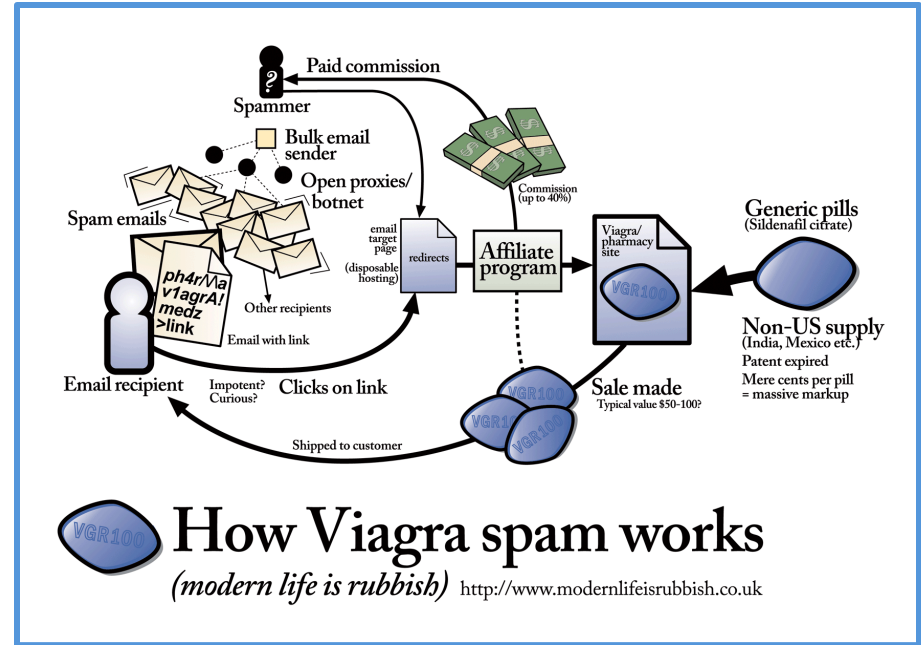
Text Classification

Task: Spam/ham detection

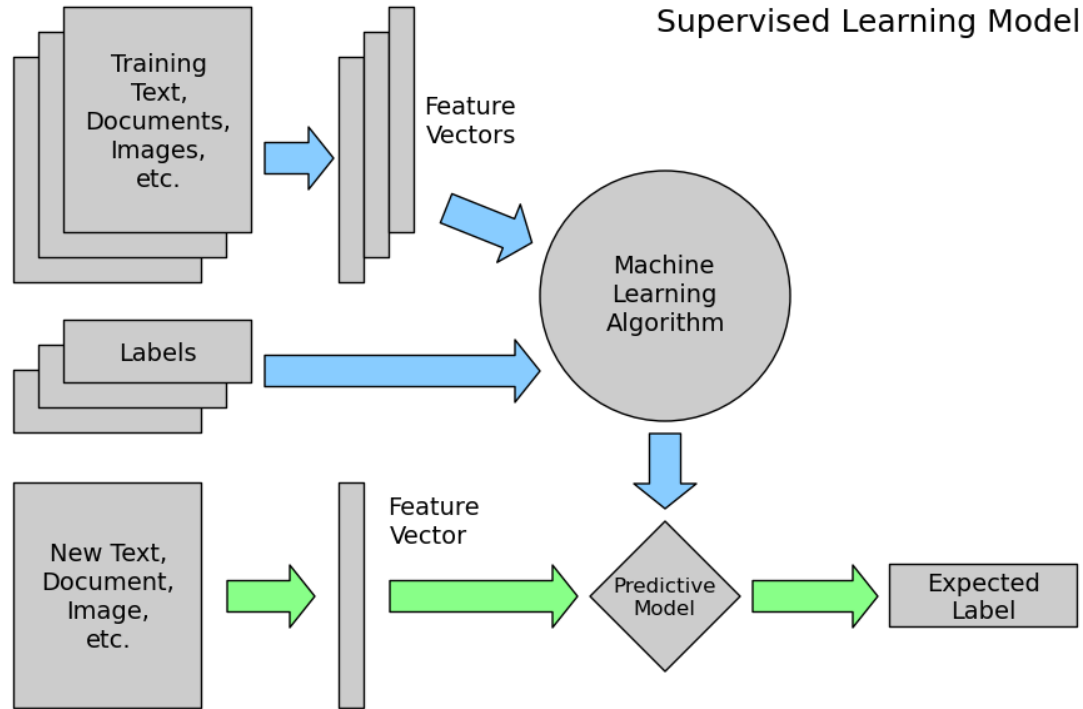


Modelling spam detection

- Input:
 - Emails
 - SMS messages
 - Facebook pages
- Predict:
 - 1 (spam)
 - 0 (ham)



Spam detection as supervised classification



Feature vectors from text



- Represent document as a multiset of words
- Keep frequency information
- Disregard grammar and word order

Feature Engineering

- Feature engineering in **Natural Language Processing (NLP)** involves transforming raw text into features that can be used by machine learning models.
- It's a crucial step in building successful NLP models because models rely on numerical features, and text is inherently unstructured.

Feature Engineering Techniques

- **Tokenization:** Splitting sentences into individual words.
"I love Python programming! " → ["I", "love", "Python", "programming", "!"]
- **Lowercasing:** Converting all text to lowercase to reduce the dimensionality of the feature space.
 - " Natural Language " → "natural language"
- **Removing Punctuation:** Eliminating punctuation that doesn't add value to the task.
 - Example: " Hello! " → " Hello "
- **Removing Stopwords:** Filtering out common words (e.g., "and", "the") that may not contribute much to the meaning.
- **Stemming and Lemmatization:** Reducing words to their root forms.
 - Stemming Example: "running", "runner" → " run "
 - Lemmatization Example: "was" → "be"

Tokenization

- Split a string (document) into pieces called tokens

"I love programming in Python!"



['I', 'love', 'programming', 'in', 'Python', '!']

```
import nltk
from nltk.tokenize import word_tokenize
# Download required resources
nltk.download('punkt')

# Example text
text = "I love programming in Python!"

# 1. Word Tokenization
word_tokens = word_tokenize(text)
print("Word Tokenization:", word_tokens)
```

```
Word Tokenization: ['I', 'love', 'programming', 'in', 'Python', '!']
```

Normalization

- Example 1: Lowercasing

"I Love Programming in Python!"



"i love programming in python!"

- Example 2: Removing Punctuation

"I love programming, in Python!"



"I love programming in Python"

```
# Example text
text = "I love programming in Python and learning NLP techniques!"

# 1. Lowercasing
text = text.lower()
print("Lowercasing: "+ text)

# 2. Removing Punctuation
text = re.sub(r'^[\w\s]', '', text)
print("Removing Punctuation: "+ text)
```

Lowercasing: i love programming in python and learning nlp techniques!
Removing Punctuation: i love programming in python and learning nlp techniques

Normalization

– Example 3: Removing Stopwords

- Stopwords are common words like

- "is", "a", "the", etc.,

"I Love Programming in Python!"



"love programming in python!"

– Example 4: Lemmatization

"The cats are running."



"The cat be run."

```
# Download required resources
nltk.download('stopwords')
nltk.download('wordnet')

# 3. Removing Stopwords
stop_words = set(stopwords.words('english'))
words = text.split()
filtered_words = [word for word in words if word not in stop_words]
# Print the results
print("Removing Stopwords: ", ' '.join(filtered_words))
# 4. Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_words]

# Join the normalized words back into a sentence
normalized_text = ' '.join(lemmatized_words)

# Print the results
print("Normalized Text:", normalized_text)
```

Removing Stopwords: love programming python learning nlp techniques
Normalized Text: love programming python learning nlp technique

Bag of Words (BoW) – Binary Indicator features

- Each word in the text is represented by a binary feature (1 if the word is present, 0 if it's absent) or by a count of its occurrences.
- Sparse representation: If the text corpus contains 1,000 unique words, each document or sentence is represented as a vector of length 1,000, with counts or binary values for each word.

Example:

- Vocabulary/Features: ["I", "love", "Python", "data", "programming"]
- Text: "I love programming in Python"
- BoW Binary Vector: [1, 1, 1, 0, 1]

Bag of Words (BoW) – Binary Indicator features

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
# Example documents
```

```
documents = [  
    "I love programming in Python!",  
    "Python programming is great.",  
    "I enjoy coding in Python and Python programming."  
]
```

```
# Create a CountVectorizer instance with stopword removal
```

```
vectorizer = CountVectorizer(stop_words='english', binary = True)
```

```
# Fit and transform the documents
```

```
X = vectorizer.fit_transform(documents)
```

```
# Convert to array and print the result
```

```
print("Feature Names:", vectorizer.get_feature_names_out())
```

```
print("Bag of Words Representation:\n", pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out()))
```

Feature Names: ['coding' 'enjoy' 'great' 'love' 'programming' 'python']

Bag of Words Representation:

	coding	enjoy	great	love	programming	python
0	0	0	0	1	1	1
1	0	0	1	0	1	1
2	1	1	0	0	1	1

Corpus: collection of documents in the context of linguistics and natural language processing (NLP)

```
# New text to be converted into the same feature vector
```

```
new_text = "I love coding with Python."
```

```
# Transform the new text using the same vectorizer
```

```
X_new = vectorizer.transform([new_text])
```

```
# Convert the new count matrix to an array and print
```

```
print("Bag of Words New X:\n", pd.DataFrame(X_new.toarray(), columns=vectorizer.get_feature_names_out()))
```

Bag of Words New X:

	coding	enjoy	great	love	programming	python
0	1	0	0	1	0	1

TF-IDF Weighting

- Term Frequency - Inverse Document Frequency (TF-IDF) captures how important a word is in a document relative to a collection of documents.
- It down-weights commonly used words and gives more importance to unique words.
 - **Example:**
 - In a corpus of documents, if the word "Python" appears frequently in one document but rarely across others, it will get a high TF-IDF score in that document.
- **$TF\text{-}IDF = TF * IDF$**
- **$IDF = \log(| docs | / | docs \text{ containing term } |)$**

TF-IDF Formula

- The TF-IDF value is calculated as follows:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

- Where:

- TF (Term Frequency): Measures how frequently a term occurs in a document. It is calculated as :

$$\text{TF}(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

- IDF (Inverse Document Frequency): Measures how important a term is throughout the entire corpus. It is calculated as:

$$\text{IDF}(t) = \log(N/\text{DF}(t))$$

Where:

N = Total number of documents in the corpus

$\text{DF}(t)$ = Number of documents containing the term t

Example Calculation

- Let's assume we have the following corpus with 3 documents:
 - "I love programming in Python"
 - "Python programming is great"
 - "I enjoy coding in Python and Python programming"

- Step 1: Calculate TF for the term "Python" in Document 1

- Document 1: 5 words total, "Python" appears 1 time.

$$\text{TF}(\text{Python}, d1) = 1/5 = 0.2$$

- Step 2: Calculate IDF (Inverse Document Frequency)

- DF (Document Frequency) for "Python":

- It appears in all 3 documents, so: $\text{DF}(\text{Python}) = 3$

- Total Number of Documents (N) = 3

- Using the IDF formula:

$$\text{IDF}(\text{Python}) = \log(N/\text{DF}(\text{Python})) = \log(3/3) = \log(1) = 0$$

TF-IDF Weighting

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Example documents
documents = [
    "I love programming in Python",
    "Python programming is great",
    "I enjoy coding in Python and Python programming"
]

# Create a TfidfVectorizer instance
vectorizer = TfidfVectorizer(stop_words='english')

# Fit and transform the documents
X = vectorizer.fit_transform(documents)

print("Feature Names:", feature_names)
print("TF-IDF Representation:\n", pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out()))
```

Feature Names: ['and' 'coding' 'enjoy' 'great' 'in' 'is' 'love' 'programming' 'python']

TF-IDF Representation:

	coding	enjoy	great	love	programming	python
0	0.000000	0.000000	0.000000	0.767495	0.453295	0.453295
1	0.000000	0.000000	0.767495	0.000000	0.453295	0.453295
2	0.516802	0.516802	0.000000	0.000000	0.305232	0.610463

```
# New text to be converted into the same feature vector
new_text = "I love coding with Python."
```

```
# Transform the new text using the same vectorizer
X_new = vectorizer.transform([new_text])
```

```
# Convert the new count matrix to an array and print
print("TF-IDF New X:\n", pd.DataFrame(X_new.toarray(), columns=vectorizer.get_feature_names_out()))
```

TF-IDF New X:

	coding	enjoy	great	love	programming	python
0	0.652491	0.0	0.0	0.652491	0.0	0.385372

Word Embeddings (like Word2Vec, GloVe, GPT, BERT)

- Word embeddings are a type of word representation that allows words to be represented as continuous vectors in a fixed-dimensional space.
- These vectors capture semantic meaning, such that words with similar meanings have vectors that are closer together in the space.
- **Word2Vec** and **GloVe** are **static** word embeddings.
 - a word like "bank" would have the same vector representation whether it's used in the context of a financial institution or the side of a river.
- **GPT** and **BERT** are **dynamic** word embeddings.
 - For example, "bank" in "river bank" and "bank" in "financial bank" will have different vector representations.

Naïve Bayes Classifier

Naïve Bayes

- It is a probabilistic classifier based on Bayes' Theorem
- It has a nice intuitive appeal
- It has been successfully used for many purposes, but it works particularly well with natural language processing (NLP) problems.

Prediction Based on Bayes' Theorem

$$P(H|\mathbf{X}) = \frac{P(\mathbf{X}|H)P(H)}{P(\mathbf{X})} = P(\mathbf{X}|H) \times P(H) / P(\mathbf{X})$$

- This can be viewed as
posteriori = likelihood x prior/evidence

Classification is to Derive the Maximum Posteriori

- Let D be a training set of tuples, and each tuple is represented by an n -D attribute vector $X = (x_1, x_2, \dots, x_n)$
- There are m class labels $C_1, C_2, \dots, C_m$ associated with D .
- Classification is to derive the maximum posteriori, i.e., the maximal $P(C_i | X)$
- This can be derived from Bayes' theorem $P(C_i | \mathbf{X}) = \frac{P(\mathbf{X} | C_i)P(C_i)}{P(\mathbf{X})}$
- Since $P(\mathbf{X})$ is constant for all classes

$$P(C_i | \mathbf{X}) = P(\mathbf{X} | C_i)P(C_i)$$

Naïve Bayes Classifier

- A simplified assumption: attributes are conditionally independent (i.e., no dependence relation between attributes): that's why it is called “naïve”

$$P(\mathbf{X} | C_i) = \prod_{k=1}^n P(x_k | C_i) = P(x_1 | C_i) \times P(x_2 | C_i) \times \dots \times P(x_n | C_i)$$

An Example

Class:

C1: Interviewed well = 'False'

C2: Interviewed well = 'True'

Data to be classified:

X = (Level =Senior,

Lang = Python,

Tweets = yes

PhD = No)

Level	Lang	Tweets	PhD	Interviewed well
Senior	Java	No	No	False
Senior	Java	No	Yes	False
Mid	Java	No	No	True
Junior	Python	No	No	True
Junior	R	Yes	No	True
Junior	R	Yes	Yes	False
Mid	R	Yes	Yes	True
Senior	Python	No	No	False
Senior	R	Yes	No	True
Junior	Python	Yes	No	True
Senior	Python	Yes	Yes	True
Mid	Python	No	Yes	True
Mid	Java	Yes	No	True
Junior	Python	No	Yes	False

Calculation

X = (Level =Senior, Lang = Python, Tweets = yes, PhD = No)

We need to compute $P(C_i | X) = P(X | C_i) * P(C_i)$

$P(C_{\text{True}})$: $P(\text{Interviewed well} = \text{"True"}) = 9/14 = 0.643$

$P(C_{\text{False}})$: $P(\text{Interviewed well} = \text{"False"}) = 5/14 = 0.357$

Compute $P(X | C_{\text{True}})$

$P(\text{Level} = \text{"Senior"} | \text{Interviewed well} = \text{"True"}) = 2/9 = 0.222$

$P(\text{Lang} = \text{"Python"} | \text{Interviewed well} = \text{"True"}) = 4/9 = 0.444$

$P(\text{Tweets} = \text{"Yes"} | \text{Interviewed well} = \text{"True"}) = 6/9 = 0.667$

$P(\text{PhD} = \text{"No"} | \text{Interviewed well} = \text{"True"}) = 6/9 = 0.667$

$P(X | C_i)$:

$P(X | \text{Interviewed well} = \text{"True"}) = 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044$

$P(X | \text{Interviewed well} = \text{"False"}) = 0.6 \times 0.4 \times 0.2 \times 0.4 = 0.019$

Compute $P(X | C_{\text{False}})$

$P(\text{Level} = \text{"Senior"} | \text{Interviewed well} = \text{"False"}) = 3/5 = 0.6$

$P(\text{Lang} = \text{"Python"} | \text{Interviewed well} = \text{"False"}) = 2/5 = 0.4$

$P(\text{Tweets} = \text{"Yes"} | \text{Interviewed well} = \text{"False"}) = 1/5 = 0.2$

$P(\text{PhD} = \text{"No"} | \text{Interviewed well} = \text{"False"}) = 2/5 = 0.4$

$P(C_i | X) = P(X | C_i) * P(C_i)$:

$P(X | \text{Interviewed well} = \text{"True"}) * P(\text{Interviewed well} = \text{"True"}) = 0.044 * 0.643 = \mathbf{0.028}$

$P(X | \text{Interviewed well} = \text{"False"}) * P(\text{Interviewed well} = \text{"False"}) = 0.019 * 0.357 = 0.007$

Since $P(C_{\text{true}} | X) > P(C_{\text{false}} | X)$, therefore X belongs to class (Interviewed well = "True")

Text Classification

Text Classification

Text	Category
"A great game"	Sports
"the election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Our goal is to build a Naïve Bayes classifier that will tell us which category the sentence " A very close game" belongs to.

Text Classification

- In our case, the probability that we wish to calculate can be calculated as:

$$p(\text{Sports} | a \text{ very close game}) = \frac{p(a \text{ very close game} | \text{Sports}) \times p(\text{Sports})}{p(a \text{ very close game})}$$

$$p(\text{Not sports} | a \text{ very close game}) = \frac{p(a \text{ very close game} | \text{Not sports}) \times p(\text{Not sports})}{p(a \text{ very close game})}$$

- Because we are only trying to find out which category (Sports or Not Sports) has a higher probability, it makes sense to discard the divisor **P(a very close game)**, and compare only:

$$p(a \text{ very close game} | \text{Sports}) \times p(\text{Sports})$$

With

$$p(a \text{ very close game} | \text{Not sports}) \times p(\text{Not sports})$$

Text Classification

- Now, we can write the probability we wish to calculate

$$p(a \text{ very close game} | \text{Sports}) = p(a | \text{Sports}) \times p(\text{very} | \text{Sports}) \times p(\text{close} | \text{Sports}) \times p(\text{game} | \text{Sports})$$

Similarly

$$p(a \text{ very close game} | \text{Not Sports}) = p(a | \text{Not Sports}) \times p(\text{very} | \text{Not Sports}) \times p(\text{close} | \text{Not Sports}) \times p(\text{game} | \text{Not Sports})$$

- But the word “close” does not exist in the category Sports,
thus $p(\text{close} | \text{Sports}) = 0$, leading to $p(a \text{ very close game} | \text{Sports}) = 0$
- Thus losing the probabilities information of other words.

Laplace smoothing

$$- p(w|c) = \frac{\text{count}(w,c)+1}{\text{count}(\text{word},c)+\text{count}(\text{word})}$$

$\text{count}(w,c)$ = Count of word w in class c

$\text{count}(\text{word},c)$ = Count of words in class c .

$\text{count}(\text{word})$ = Count all the possible words in the dataset

– **Now we can calculate the probability again :**

$$p(\textit{close}|\textit{Sports}) = \frac{(0 + 1)}{(11 + 14)}$$

Calculation

$$p(\text{Sports} | \text{a very close game}) = ?$$

$$p(\text{Not Sports} | \text{a very close game}) = ?$$

Let $Z = \text{A Very Close Game}$; $S = \text{Sport}$; $NS = \text{Non Sport}$

Therefore probability of "A Very Close Game " being a Sport $\Rightarrow P(S | Z) = P(Z | S) P(S)$

$$P(A | S) \times P(V | S) \times P(C | S) \times P(G | S) \times P(S) = [3/25 \times 2/25 \times 1/25 \times 3/25] \times 3/5 = 0.0004608 \times 0.6 = 0.000027648$$

And the probability of "A Very Close Game " being a Non Sport $\Rightarrow P(NS | Z) = P(Z | NS) P(NS)$

$$P(A | NS) \times P(V | NS) \times P(C | NS) \times P(G | NS) \times P(NS) = [2/23 \times 1/23 \times 2/23 \times 1/23] \times 2/5 = 0.00001429 \times 0.4 = 0.00000571$$

Then "a very close game" is belong to the Sports class

w	$p(w \text{Sports})$	$p(w \text{Not Sports})$
a	$\frac{(2 + 1)}{(11 + 14)}$	$\frac{(1 + 1)}{(9 + 14)}$
very	$\frac{(1 + 1)}{(11 + 14)}$	$\frac{(0 + 1)}{(9 + 14)}$
close	$\frac{(0 + 1)}{(11 + 14)}$	$\frac{(1 + 1)}{(9 + 14)}$
game	$\frac{(2 + 1)}{(11 + 14)}$	$\frac{(0 + 1)}{(9 + 14)}$

SMS spam detection

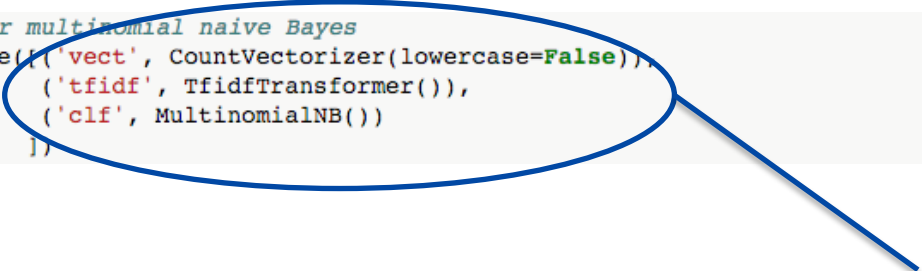
Label	Message snippet
Ham	Go until jurong point, crazy.. Available only ...
Spam	Ok lar... Joking wif u oni...
Ham	U dun say so early hor... U c already then say...
Spam	FreeMsg Hey there darling it's been 3 week's n...

- 425 SMS spam messages from UK Grumbletext web forum
- 3,375 ham randomly chosen from NUS SMS corpus (students)
- 450 ham from somebody's PhD thesis
- 322 spam and 1,002 ham from SMS Spam Corpus

Use scikit-learn Pipeline to manage cross validation

Scikit-learn Pipelines provide a mechanism for fitting and predicting a sequence of components. This is good practice to avoid data leakage.

```
# Pipeline for multinomial naive Bayes
mnb = Pipeline([('vect', CountVectorizer(lowercase=False)),
                 ('tfidf', TfidfTransformer()),
                 ('clf', MultinomialNB())
                ])
```



1. Convert string to a bag-of-words token vector.
2. Transform vector counts using TFIDF weighting.
3. Train/predict using multinomial naïve Bayes.

Exercise: SMS spam filtering

- Build a spam filter
 -  code cell under “Download data from UCI ML data repo”
 -  code cell under “Read and profile data using Pandas”
 -  code cell under “Text feature extraction”
 -  code cell under “Build pipelines and choose parameters”
- Exercises
 - Which is better: support vector or multinomial naïve Bayes classifier?
 - Handling class imbalance in support vector classifier
 - Generalization, data size, features

Text-driven forecasting

Text-driven forecasting

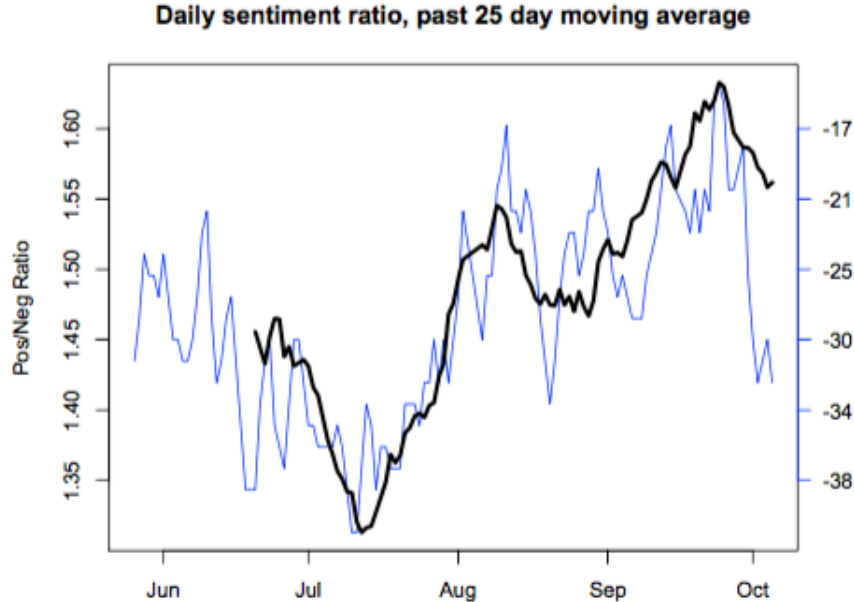
Given a body of text T pertinent to a social phenomenon, make a concrete prediction about a measurement M of that phenomenon, obtainable only in the future.

<http://www.cs.cmu.edu/~nasmith/papers/smith.whitepaper10.pdf>

Some text-driven forecasting tasks

- Predict box office gross for films
 - T : description, script, reviews, etc
 - M : how much the film earns at the box office
- Predict volatility of a stock
 - T : annual report, etc
 - M : volatility over the following year
- Predict blog reader behaviour
 - T : political blog posts, etc
 - M : number of reader comments

Predicting public opinion from tweets



- T: tweets mentioning the word “economy”
- M: Gallup’s economic confidence index (blue)
- Predictions (black) closely track Gallup’s polling data

<https://www.aacai.org/ocs/index.php/ICWSM/ICWSM10/paper/viewFile/1536/1842>

Exercise: Predicting box office gross

- Build model to forecast gross
 -  code cell under “Download reviews and movie gross data”
 -  code cell under “Select parameters for support vector regression”
- Exercises
 - Which parameters are best?
 - Is the model a good fit?
 - How could we improve the experimental setup?

Review

Review: Unstructured data

Objective

Learn machine learning tools in Python for text categorization and forecasting.

Lecture

- Text pre-processing
- Naïve Bayes
- Text-driven forecasting

Readings

- Data Science from Scratch, Ch. 13
- Doing Data Science, Ch. 4

Exercises

- Spam detection
- Predicting box office returns
- Information extraction

TODO in W10

- Analyse and characterise results

Natural language processing (not examinable)

- Manning and Schutze. Foundations of statistical NLP.
<http://nlp.stanford.edu/fsnlp/>
- Jurafsky and Martin. Speech and language processing.
<https://web.stanford.edu/~jurafsky/slp3/>
- Bird et al. Natural language processing with Python.
<http://www.nltk.org/book/>
- CMU seminar on text-driven forecasting.
<http://www.cs.cmu.edu/~nasmith/TDF/>

Thank you